

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования «МИРЭА, Российский технологический университет»

РТУ МИРЭА

Институт перспективных технологий и индустриального программирования

Кафедра индустриального программирования

Практическая работа №10

по дисциплине Технологии разработки приложений на базе фреймворков

Административная панель Django и работа с данными через ORM

Группа: ЭФБО-04-24

Выполнил: Валекжанин Владимир Сергеевич

Проверил:

Ящун Т. В.

Москва

2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....4
1 МАТЕРИАЛЫ ЯНДЕКС ПРАКТИКУМА.....5
2 НАСТРОЙКА АДМИНИСТРАТИВНОЙ ПАНЕЛИ.....6
3 ИЗМЕНЕНИЯ МОДЕЛЕЙ И МИГРАЦИЙ.....8
4 РАБОТА С ДАННЫМИ ЧЕРЕЗ ORM.....10
5 АНАЛИЗ SQL И DJANGO DEBUG TOOLBAR.....12
6 РЕЗУЛЬТАТ И ПРОВЕРКА.....14
ВЫВОД.....16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....17

ВВЕДЕНИЕ

Цель работы по методичке: освоить административную панель Django и углубить навыки работы с данными через ORM. В проекте Concerty это означает: зарегистрировать все модели в админ-зоне и настроить её под задачи контент-менеджера, заменить булево поле is_cancelled концертов на полноценный статус жизненного цикла, добавить площадкам слаг и вычисляемое поле is_large, подключить django-debug-toolbar и анализировать реальные SQL-запросы страниц. Материалы курса «Бэкенд-разработка» дают приёмы тонкой настройки админ-зоны и построения запросов [1].

1 МАТЕРИАЛЫ ЯНДЕКС ПРАКТИКУМА

Выполнены материалы разделов «Административная панель Django» и «Работа с данными через ORM» курса «Бэкенд-разработка». Перечень тем и отметки о выполнении приведены ниже.

Таблица 1 — Материалы курса «Бэкенд-разработка» по теме работы

Тема	**Отметка о выполнении**
Админ-зона Django: суперпользователь и регистрация моделей	Выполнено
Локализация и перевод	Выполнено
Тонкая настройка админ-зоны	Выполнено
Django ORM, получение информации из БД: Debug Toolbar	Выполнено
SELECT и FROM через ORM	Выполнено
filter() и exclude()	Выполнено
Q-объекты	Выполнено
order_by(), limit и offset	Выполнено
Получение отдельных объектов	Выполнено
Запросы к связанным моделям	Выполнено
Доработка проекта	Выполнено
Финальный проект спринта: публикации для Блогикум	Выполнено

Все двенадцать тем в таблице 1 отмечены как выполненные, приёмы повторены на данных проекта Concerty.

2 НАСТРОЙКА АДМИНИСТРАТИВНОЙ ПАНЕЛИ

В административной панели зарегистрированы все четыре модели: Performer, Venue, Program и Concert. Для каждой модели написан класс ModelAdmin с параметрами списка записей. Основной приём настройки: вывести в список только нужные поля, дать ссылки на ключевые колонки, добавить фильтры и поиск, включить постраничный вывод по 20 записей [2].

Таблица 2 — Параметры ModelAdmin по моделям проекта

Модель	**List display**	**List display links**	**List filter**	**Search fields**
Concert	id, title, date, performer, venue, status	title	status, date, venue	title, performer__name, venue__name
Venue	id, name, city, capacity, slug, is_large	name	city, is_large	name, city
Performer	id, name, genre, rating	name	genre	name, genre
Program	id, name, duration, age_limit	name	age_limit	name

Параметры списков приведены в таблице 2. У ConcertAdmin дополнительно заданы date_hierarchy по полю date (навигация по датам справа от списка), ordering по дате и идентификатору и list_select_related для связанных объектов. Код настройки показан в листинге ниже.

Листинг 1 — Настройка ConcertAdmin в concerts/admin.py

```
@admin.register(Concert)
class ConcertAdmin(admin.ModelAdmin):
    list_display = ("id", "title", "date", "performer", "venue", "status")
    list_display_links = ("title",)
    list_filter = ("status", "date", "venue")
    search_fields = ("title", "performer__name", "venue__name")
    # Связанные объекты загружаются одним запросом.
    list_select_related = ("performer", "venue", "program")
    date_hierarchy = "date"
    ordering = ("date", "pk")
    list_per_page = 20
```

Фрагмент настройки списка концертов приведён в листинге 1. Поиск по связанным моделям работает через обращение через внешний ключ performer__name.

Листинг 2 — Инлайн концертов в карточке площадки в venues/admin.py

```
class ConcertInline(admin.TabularInline):
    model = Concert
    fields = ("title", "performer", "program", "date", "status")
    readonly_fields = fields
    extra = 0
    can_delete = False


@admin.register(Venue)
class VenueAdmin(admin.ModelAdmin):
    ...
    inlines = (ConcertInline,)
```

В карточки площадки, исполнителя и программы добавлен инлайн-просмотр связанных концертов в листинге 2. Поля инлайна только для чтения, удаление отключено: внешние ключи Concert защищены PROTECT, поэтому менять список концертов из чужой карточки нельзя, а удалить их и вовсе не выйдет.

Локализация: в пяти приложениях заданы AppConfig с русскими verbose_name, у моделей переведены названия и множественные формы, в settings.py установлен LANGUAGE_CODE ru-RU. На странице редактирования площадки появилась ссылка «Смотреть на сайте», она строится методом get_absolute_url модели Venue.

3 ИЗМЕНЕНИЯ МОДЕЛЕЙ И МИГРАЦИЙ

Булево поле is_cancelled заменено на поле status с вариантами выбора Concert.Status (TextChoices): SCHEDULED, CANCELLED, COMPLETED. Поверх поля построены менеджер ConcertManager и класс запросов ConcertQuerySet с методами scheduled, cancelled, completed и for_date. Код приведён ниже.

Листинг 3 — Статусы и менеджер концертов в concerts/models.py

```
class ConcertQuerySet(models.QuerySet):
    def scheduled(self):
        return self.filter(status=Concert.Status.SCHEDULED)

    def cancelled(self):
        return self.filter(status=Concert.Status.CANCELLED)

    def completed(self):
        return self.filter(status=Concert.Status.COMPLETED)

    def for_date(self, concert_date):
        return self.filter(date=concert_date)

class Concert(models.Model):
    class Status(models.TextChoices):
        SCHEDULED = "SCHEDULED", "Запланирован"
        CANCELLED = "CANCELLED", "Отменён"
        COMPLETED = "COMPLETED", "Завершён"
    ...
    status = models.CharField(max_length=20, choices=Status.choices, ...)
    objects = ConcertManager()
```

Менеджер и статусы в листинге 3 дают читаемые вызовы Concert.objects.scheduled() вместо повторяющихся filter(status=...). Порядок миграций описан в документации Django [3].

Переход выполнен тремя миграциями: 0002_concert_status добавляет поле status, 0003_migrate_concert_status переносит данные (истина в is_cancelled стала CANCELLED, прошедшие даты стали COMPLETED, остальное SCHEDULED), 0004_remove_concert_is_cancelled удаляет старое поле. У площадок миграции: 0002_venue_is_large_venue_slug добавляет слаг и GeneratedField is_large, 0003_fill_venue_slugs заполняет слаги у существующих записей, 0004_alter_venue_slug делает слаг уникальным.

Поле Venue.slug заполняется автоматически из названия при первом сохранении, кириллица сохраняется. Для маршрута площадки зарегистрирован конвертер uslug на основе UnicodeSlugConverter, потому что стандартный конвертер slug кириллицу не пропускает. Вместимость площадки не меньше 500 мест отражает вычисляемое поле is_large: GeneratedField с выражением Q по полю capacity__gte (вместимость не меньше 500) и параметром db_persist равным True, значение хранится в базе и обновляется самой СУБД [4].

4 РАБОТА С ДАННЫМИ ЧЕРЕЗ ORM

Представления построены на запросах ORM и описаны в документации Django [5, 6]. Список концертов фильтруется по статусу, дате, площадке и исполнителю через filter, поиск собирается Q-объектами с icontains по названию и имени исполнителя. Сортировка идёт только по белому списку SORT_FIELDS, параметр из адресной строки подставляется в order_by без возможности SQL-инъекции. Пагинация Paginator выводит по шесть записей на страницу, параметры фильтров сохраняются при переходе по страницам шаблонным тегом querystring. Детальная страница концерта и карточка площадки берут объекты через get_object_or_404 с select_related. Страница статистики использует values, annotate и aggregate: счётчики, суммы и средние по трём статусам концертов.

Подбор свободных площадок на дату строится на exclude: идентификаторы площадок с запланированными концертами на выбранную дату убираются из общего списка. Отменённые и завершённые концерты площадку не блокируют. Проверка логики в Django shell приведена ниже.

Листинг 4 — Транскрипт проверки ORM в Django shell

```
(a) scheduled: 4
(a) cancelled: 0
(a) completed: 11
(b) for_date 2026-11-14 -> 1
(c) venue 11 absent from free list on 2026-11-14 : True
(c) completed concert 5 venue 4 present in free list: True
(c2) cancelled concert 160 venue 9 present in free list: True
(c2) temp cancelled concert deleted
(d) slug: adrenaline-stadium
(d) url: /venues/adrenaline-stadium/
(e) capacity=499 is_large: False
(e) capacity=500 is_large: True
(e) temp venue deleted
```

Транскрипт в листинге 4 подтверждает: менеджер считает концерты по статусам, for_date выбирает концерты даты, площадка с запланированным концертом исключена из списка свободных, площадки с завершённым и отменённым концертами в списке присутствуют. Слаг и адрес первой

площадки отвечают get_absolute_url, а GeneratedField меняет is_large на границе вместимости 500 мест.

5 АНАЛИЗ SQL И DJANGO DEBUG TOOLBAR

django-debug-toolbar 8.0 подключён четырьмя правками: приложение debug_toolbar добавлено в INSTALLED_APPS, DebugToolbarMiddleware вставлен в MIDDLEWARE, INTERNAL_IPS ограничен адресами localhost и 127.0.0.1, маршрут __debug__/ подключается в urls.py только при DEBUG равном истине. Зависимости зафиксированы в pyproject.toml, uv.lock и requirements.txt. Панель показывает SQL-запросы страницы, их время и план выполнения [7].

Число запросов страниц измерено через django.test.utils.CaptureQueriesContext, итоги приведены ниже.

Таблица 3 — Число SQL-запросов на страницу в тестовом клиенте

Страница	**Код ответа**	**Число запросов**
/concerts/	200	5
/concerts/8/	200	2
/venues/	200	3
/venues/available/?date=2026-11-14	200	2

Числа в таблице 3 сняты без панели инструментов, лишние запросы шаблонов отсутствуют. Главный SELECT детальной страницы концерта показан ниже.

Листинг 5 — SQL детальной страницы концерта с JOIN от select_related

```
SELECT "concerts_concert"."id", "concerts_concert"."title",
 "concerts_concert"."performer_id", "concerts_concert"."venue_id",
 "concerts_concert"."program_id", "concerts_concert"."date",
 "concerts_concert"."expected_guests",
"concerts_concert"."base_ticket_price",
 "concerts_concert"."status", "performers_performer"."id",
 "performers_performer"."name", "performers_performer"."genre",
 "performers_performer"."rating", "venues_venue"."id", "venues_venue"."name",
 "venues_venue"."city", "venues_venue"."capacity", "venues_venue"."slug",
 "venues_venue"."is_large", "programs_program"."id",
"programs_program"."name",
 "programs_program"."duration", "programs_program"."age_limit"
FROM "concerts_concert"
INNER JOIN "performers_performer"
 ON ("concerts_concert"."performer_id" = "performers_performer"."id")
INNER JOIN "venues_venue"
 ON ("concerts_concert"."venue_id" = "venues_venue"."id")
INNER JOIN "programs_program"
```

```
 ON ("concerts_concert"."program_id" = "programs_program"."id")
WHERE "concerts_concert"."id" = 8 LIMIT 21
```

Запрос в листинге 5 выполняется один раз: select_related собирает исполнителя, площадку и программу тремя JOIN в том же SELECT. Без select_related каждая связь дала бы отдельный запрос, и страница выборки из N концертов породила бы один основной и по три дополнительных запроса на каждый концерт, то есть рост числа запросов линейный по N, это известная проблема N плюс один запрос. План запроса менеджера Concert.objects.scheduled() подтверждает простой обход таблицы concerts_concert и сортировку через временное B-дерево по полю ordering модели.

Листинг 6 — План запроса Concert.objects.scheduled() по .explain()

```
3 0 216 SCAN concerts_concert
19 0 0 USE TEMP B-TREE FOR ORDER BY
```

План запроса приведён в листинге 6: SQLite читает таблицу целиком, индексов на фильтруемые поля в учебной базе не создано, для объёма данных проекта этого достаточно.

6 РЕЗУЛЬТАТ И ПРОВЕРКА

1. Зарегистрированы четыре модели в админ-зоне с фильтрами, поиском, иерархией по дате и постраничным выводом по 20 записей.
2. Добавлены инлайн-формы концертов в карточки площадки, исполнителя и программы, удаление в инлайнах отключено из-за PROTECT.
3. Поле is_cancelled заменено статусами TextChoices с менеджером ConcertQuerySet, данные перенесены миграцией.
4. Площадки получили автоматические слаги с поддержкой кириллицы и GeneratedField is_large на границе 500 мест.
5. Контекстный процессор concert_context передаёт site_name и число концертов в base.html и навигацию, интерфейс полностью локализован.
6. Подключён django-debug-toolbar, SQL-запросы страниц проанализированы, проблема N плюс 1 не выявлена.

6.1 Проверка команд

Команды проверки запущены из корня проекта, их результаты приведены ниже.

Таблица 4 — Результаты проверок проекта

Команда	**Результат**
python manage.py check	System check identified no issues (0 silenced)
python manage.py makemigrations --check	No changes detected
python manage.py showmigrations	Отметка [X] у миграций 0002-0004 приложений concerts и venues
pytest -q	246 passed in 8.16s
flake8 . --extend-exclude=venv	Замечаний нет, код выхода 0

Все проверки в таблице 4 проходят без ошибок, миграции применены к базе.

6.2 Пользовательские сценарии

Таблица 5 — Проверенные пользовательские сценарии

Сценарий	**Результат**
Главная, списки концертов и площадок	Открываются, данные из ORM
Фильтры и поиск списка концертов	Работают, параметры сохраняются
Пагинация по шесть записей	Переключение страниц сохраняет фильтры
Несуществующий слаг площадки	Страница 404
Страница статистики	Агрегаты по трём статусам концертов
Подбор площадок на дату	Список свободных площадок корректен

Сценарии в таблице 5 покрыты автотестами tests/test_pr10.py и ручной проверкой.

6.3 Git

Работа выполнена в ветке main. Изменения ПР10 зафиксированы тремя коммитами.

Таблица 6 — Коммиты ПР10 в ветке main

Коммит	**Сообщение**
a4afe54	статусы концертов, слаги площадок, GeneratedField is_large, миграции данных
295107f	админ-зона с инлайнами, контекстный процессор, локализация, Debug Toolbar
54da89d	тесты, README и зависимости (debug-toolbar, pypdf)

Список коммитов приведён в таблице 6. Файлы отчёта добавляются отдельным коммитом после сборки.

ВЫВОД

Административная панель проекта настроена для контент-менеджера: модели зарегистрированы, списки отфильтрованы и переведены, концерты видны внутри карточек связанных сущностей. Модельный слой усилен статусами жизненного цикла, менеджером запросов, слагами и вычисляемым полем GeneratedField, переход данных выполнен миграциями без потерь. ORM даёт фильтрацию, поиск, сортировку и пагинацию, а Debug Toolbar и замер запросов подтвердили отсутствие лишних обращений к базе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Яндекс Практикум Бэкенд-разработка [Электронный ресурс]. — URL: https://practicum.yandex.ru/backend-developer/ (дата обращения: 09.11.2026).
2. Django Software Foundation The Django admin site [Электронный ресурс]. — URL: https://docs.djangoproject.com/en/5.2/ref/contrib/admin/ (дата обращения: 09.11.2026).
3. Django Software Foundation Migrations [Электронный ресурс]. — URL: https://docs.djangoproject.com/en/5.2/topics/migrations/ (дата обращения: 09.11.2026).
4. Django Software Foundation GeneratedField [Электронный ресурс]. — URL: https://docs.djangoproject.com/en/5.2/ref/models/fields/#generatedfield (дата обращения: 09.11.2026).
5. Django Software Foundation Making queries [Электронный ресурс]. — URL: https://docs.djangoproject.com/en/5.2/topics/db/queries/ (дата обращения: 09.11.2026).
6. Django Software Foundation QuerySet API reference [Электронный ресурс]. — URL: https://docs.djangoproject.com/en/5.2/ref/models/querysets/ (дата обращения: 09.11.2026).
7. Jazzband django-debug-toolbar documentation [Электронный ресурс]. — URL: https://django-debug-toolbar.readthedocs.io/en/latest/ (дата обращения: 09.11.2026).